

DML: DATA MANIPULATION LANGUAGE

- In SQL sono previsti tre comandi per modificare il database:
 - INSERT**: popolamento tabelle
 - DELETE**: rimozione di righe
 - UPDATE**: cambiamento di valori nelle righe delle tabelle

DELETE

- Il comando **DELETE** rimuove una o più tuple da una relazione.

```
DELETE
FROM <NOME TABELLA>
WHERE <CONDIZIONE>;
```

- Cancella da <NOME TABELLA> tutte le righe che soddisfano <CONDIZIONE>
- Se la clausola **WHERE** è omessa allora è come se ci fosse la clausola

WHERE TRUE

e vengono cancellate tutte le righe della tabella

- NB**: Il comando DELETE non rimuove la tabella
 - Il comando DROP <Nome Tabella> lo fa

Cosa succede coi vincoli di integrità referenziale?

VENI SOTTO ↓

IL COMANDO DELETE - ESEMPI

```
DELETE FROM EMPLOYEE
WHERE LNAME='Brown';
```

```
DELETE FROM EMPLOYEE
WHERE DNO IN (SELECT DNUMBER
FROM DEPARTMENT
WHERE DNAME='Research');
```

query annidata nella clausola WHERE

IL COMANDO INSERT

- Il comando **INSERT INTO** inserisce nuove righe in una relazione.

```
INSERT INTO <Nome Tabella> [ElencoAttributi]
VALUES (ListaValori);
```

oppure

```
INSERT INTO <NomeTabella> [ElencoAttributi]
SELECT ElencoAttributi
FROM Espressione;
```

IL COMANDO INSERT (2)

INSERT INTO <Nome Tabella> [ElencoAttributi]
VALUES (ListaValori);

1. Se l'elenco degli attributi viene omesso, allora si intendono riempire i campi degli attributi in **ordine di definizione della tabella**
2. Se specifico l'elenco degli attributi, allora l'elenco deve includere tutti gli attributi TOTALI (**NOT NULL**). Decidiamo noi l'ordine.
3. Gli attributi PARZIALI non presenti, assumeranno il valore NULL o il valore di DEFAULT, se questo è specificato

```
INSERT INTO EMPLOYEE (FNAME, LNAME, SSN)
VALUES ('Richard', 'Marini', '654765876');
```

```
INSERT INTO EMPLOYEE
VALUES ('Richard', 'K', 'Marini', '654765876', '30-DEC-52', '98 Oak Forest, Katy, TX',
'M', 37000, '987654321', 4);
```

IL COMANDO INSERT - ESEMPIO

- Creare una **tabella derivata (temporanea)** che ha nome, numero di impiegati e salari totali per ciascun dipartimento:

```
CREATE TABLE DEPTS_INFO (DEPT_NAME VARCHAR(15),
NO_OF_EMPS INTEGER,
TOTAL SAL INTEGER);
```

CREATE TABLE

```
INSERT INTO DEPTS_INFO (DEPT_NAME, NO_OF_EMPS, TOTAL SAL)
```

```
SELECT DNAME, COUNT(*), SUM(SALARY)
FROM DEPARTMENT, EMPLOYEE
WHERE DNUMBER=DNO
GROUP BY DNAME;
```

separazione della
nuova tabella
(nome + attributi)

join

questo è un
risultato viene
inserito nella
nuova tabella

- Eventuali aggiornamenti successivi non influenzano la tabella
Per aggiornarle, è invece necessario definire una **view**.

IL COMANDO UPDATE

- Il comando **UPDATE** permette di **modificare** valori in una relazione:

UPDATE <NomeTabella> [alias]

SET <NomeAttributo1>=<Espressione> [, <NomeAttributoN>=<EspressioneN>]

[**WHERE** <condizione>]

1. Si lavora su NomeTabella
2. Si modificano le righe che soddisfano <condizione>
3. Per ogni riga si modificano gli attributi della clausola SET con il valore calcolato dall'espressione

IL COMANDO UPDATE - ESEMPIO

```
UPDATE PROJECT
SET PLOCATION='Bellaire', DNUM=5
WHERE PNUMBER=10;
```

aumento del 10% il salario degli
impiegati del dip. 'Research'

```
UPDATE EMPLOYEE
SET SALARY=SALARY * 1.1
WHERE DNO IN (SELECT DNUMBER
FROM DEPARTMENT
WHERE DNAME='Research');
```

restituisce il numero
del dipartim. 'Research'

ESEMPIO

Magazzino (CodArticolo, Descrizione, Quantità)

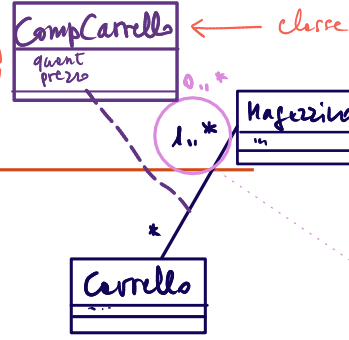
Ordine (CodOrdine, Data, PIVA, DataInvio)

CompOrdine(CodOrdine*, CodArticolo*, Quantità, Prezzo)

Carrello(CodCarrello, Data, PIVA, Completo)

CompCarrello(CodCarrello*, CodArticolo*, Quantità, Prezzo)

ESERCIZIO: fare `reverse engineering` e produrre un diagramma concettuale (ER/UML) da cui si deriva questo schema logico.



CompOrdine è envelope a CompCarrello

Operazioni che devono essere fatte

Quando il carrello è completo

- Trasformo il carrello in ordine
 - Creare un nuovo ordine con la struttura del carrello
 - Rimuovere il carrello
- Aggiornare le scorte nel magazzino

Tabella che rappresenta la associazione tra Carrello e Magazzino (multi-a-multi)

I casi d'uso del sistema informativo (nella nostra specifica immaginaria per questo esercizio) prevede che possa essere un carrello vuoto. Quindi il vincolo 1..* è troppo restrittivo ed è bene rimpiazzarlo con 0..* (denotato altresì con *) per ammettere carrelli vuoti.

In generale è bene fare attenzione a non imporre vincoli troppo restrittivi; è bene avere sempre come riferimento i casi d'uso.

Operazioni che devono essere fatte

Quando il carrello è completo

- Trasformo il carrello in ordine
 - Creare un nuovo ordine con la struttura del carrello
 - Rimuovere il carrello
- Aggiornare le scorte nel magazzino

Che operazioni di Manipolazione devo fare?

In che ordine devo fare le operazioni di manipolazione?

ESEMPIO (2)

Quando il carrello è completo

- Trasformo il carrello in ordine
 - Creare un nuovo ordine con la struttura del carrello
(INSERT in ORDINE e COMPOORDINE)
 - Rimuovere il carrello
(DELETE in CARRELLO e COMPCARRELLO)
- Aggiornare le scorte nel magazzino
(UPDATE su MAGAZZINO)

Come sempre, parte della logica di queste operazioni potrebbe essere delegata alla applicazione.

ESEMPIO (3)

Creare un nuovo ordine con la struttura del carrello (Lavoriamo con il carrello con il codice 'XYZ')

1) INSERT INTO Ordine (CodO, Data, PIVA, DataInvio)

```

(SELECT C.CodO, Data, PIVA, NULL
FROM Carrello C
WHERE C.CodO = 'XYZ')
    
```

Il risultato di questa SELECT viene inserito in Ordine

Oppure

INSERT INTO Ordine (CodO, Data, PIVA)

```

(SELECT C.CodO, Data, PIVA
FROM Carrello C
WHERE C.CodO = 'XYZ')
    
```

Omissione di DataInvio ⇒ DataInvio viene impostato a NULL

Data impostata a NULL

ESEMPIO (4)

- Creare un nuovo ordine con la struttura del carrello (Lavoriamo con il carrello con il codice 'XYZ')

2) **INSERT INTO** CompOrdine

```
(SELECT *  
FROM CompCarrello C  
WHERE C.CodC = 'XYZ')
```

CompOrdine : contenuto dell'ordine

ESEMPIO (5)

Cancella tutti gli elementi del carrello XYZ da CompCarrello, poi cancella il medesimo carrello XYZ da Carrello.

Lo SCHEMA
↓
È ALGIBO

2) **DELETE**

```
FROM CompCarrello C  
WHERE C.CodC = 'XYZ'
```

①

```
DELETE  
FROM Carrello C  
WHERE C.CodC = 'XYZ'
```

②

Si potrebbe fare il contrario?
Dipende da quello che abbiamo
definito
con la FK

ESEMPIO

Magazzino (CodArticolo, Descrizione, Quantità)
Ordine (CodOrdine, Data, PIVA, DataInvio)
CompOrdine (CodOrdine*, CodArticolo*, Quantità, Prezzo)
Carrello (CodCarrello, Data, PIVA, Completo)
CompCarrello (CodCarrello*, CodArticolo*, Quantità, Prezzo)

ESEMPIO (6)

- Aggiornare le scorte nel magazzino
 - Per ogni articolo nell'ordine 'XYZ' devo rimuoverne la quantità venduta nel magazzino

b) **UPDATE** Magazzino M

```
SET M.Quantità = M.Quantità - (SELECT H.Quantità  
FROM CompOrdine H  
WHERE H.CodO = 'XYZ' AND H.CodA = M.CodA)  
WHERE M.CodA IN (SELECT C.CodA  
FROM CompOrdine C  
WHERE C.CodO = 'XYZ')
```

TRANSACTION

Un **COMMIT** è la azione di consolidamento dei cambiamenti effettuati sul DB con le operazioni di modifica/inserimento/cancellazione.

Sequenza di operazioni tra due COMMIT.

ROLLBACK: ritornare allo stato del precedente COMMIT.

BEGIN TRANSACTION

INSERT...

UPDATE...

DELETE...

procedure...

COMMIT

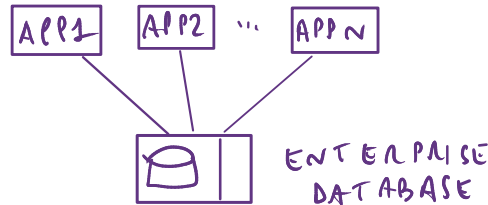
END TRANSACTION

Questo ci consente di differire la verifica dei vincoli alla fine della sequenza di operazioni (transazione).

TRANSAZIONE: operazione atomica, cioè essa è eseguita o tutta o per nulla, essendo essa costituita in generale da più operazioni sui dati specificate in SQL.

TRIGGER

- Un trigger è una di procedura/funzione di manutenzione della base di dati
- A differenza delle procedure standard, che sono chiamate in maniera esplicita, un trigger non è invocato.
- La loro attivazione è scatenata da una serie di eventi di diversa natura che avvengono nella base di dati
- Questi eventi scatenano una serie di operazioni che servono per mantenere la base di dati
 - Gli eventi che ci interessano, in particolare sono quelli che vanno ad alterare la base di dati
 - INSERT
 - UPDATE
 - DELETE



TRIGGER

ECA *evento - condizione - azione*

- Un tipico trigger è definito come una la composizione di tre componenti
 - L'**EVENTO** che lancia il trigger. Solitamente operazioni di aggiornamenti espliciti alla base di dati.
 - Inserimento di un record *innesca*
 - Cancellazione di un record
 - Modifica di un record
 - Eventi innescati temporalmente o altri eventi esterni
 - La **CONDIZIONE** che determina se l'azione della regola deve essere eseguita
 - Quando si verifica l'evento si può valutare una condizione (facoltativa - in mancanza l'azione è effettuata ogni volta che si verifica l'evento)
 - Se la condizione è specificata allora l'azione è eseguita ogni volta che questa da valore TRUE
 - L'**AZIONE** da intraprendere
 - Di solito l'azione corrisponde ad una sequenza di istruzioni SQL ma può essere anche l'esecuzione di una transazione, di una procedura o di un programma esterno

USARE I TRIGGER CON ATTENZIONE

L'interazione tra le esecuzioni di diversi trigger può essere complessa e i suoi effetti difficili da controllare.

CREAZIONE TRIGGER

- Quando voglio creare un trigger, chiaramente devo andare a definire i parametri del trigger:
- Quale evento scatena la reazione?
 - INSERT
 - DELETE
 - UPDATE
 - Quando voglio scatenare la reazione? Prima o dopo l'evento?
 - Dopo: C'è stato, ad esempio, un UPDATE e voglio propagare l'evento
 - Prima: Ci sarà, ad esempio, una DELETE e voglio eseguire dei comandi prima della cancellazione
 - Condizioni di Filtraggio che mi permettono di definire precisamente quale è l'evento che scatena la reazione
 - Operazione globale/per ciascuna riga
 - Reazione

segnatura

SIGNATURE TRIGGER

DDL : definizione trigger

CREATE TRIGGER <nomeTrigger>

[AFTER/BEFORE/INSTEAD OF] <operazione>

[FOR EACH ROW]

[WHEN <condizione>] ← *CONDIZ.*

BEGIN

<CORPO DELLA PROCEDURA> ← *AZIONE*

END

```
INSERT ON <tabella>
DELETE ON <tabella>
UPDATE ON <tabella> OF <ATTRIBUTO>
```

Queste sintassi non vale per POSTGRESQL

CONTROL CLASSES

DOMAIN CLASSES

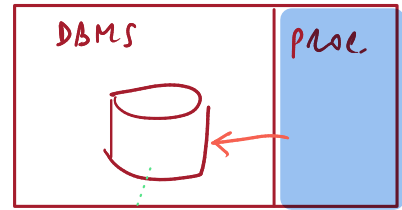
BOUNDARY

BOUNDARY CLASSES

APPLICATION LOGIC

DATABASE

DATABASE ACCESS CLASSES



CONSISTENZA DELLA BASE DI DATI RISPETTA AI VINCOLI

VINCOLO

Es. La somme dei salari di ogni dip. deve essere inferiore al budget per detto dipartimento.

DOVE LO SI IMPLEMENTA?

- NEL DBMS (VINCOLI DI BASE)
- NEL DBMS (CON TRIGGER)
- NELL'APPLICAZIONE
 - PARTI DB
 - PARTI LOGICA
 - PARTI BOUNDARY

SINTASSI

- **CREATE TRIGGER** <nomeTrigger>
 - Specifica il nome del trigger
- **[AFTER/BEFORE/INSTEAD OF]** <operazione>
 - **AFTER** indica che la regola deve essere attivata dopo il verificarsi degli eventi
 - **BEFORE** indica che la regola deve essere attivata prima del verificarsi degli eventi
 - **INSTEAD OF** esegue il trigger invece di eseguire l'evento (aggiornamenti su viste)
- **[FOR EACH ROW]**
 - Indica che la regola verrà attivata una volta per ogni riga influenzata dall'evento
 - In assenza della clausola [FOR EACH ROW] l'azione verrà eseguita una sola volta, anche se più righe sono state influenzate dall'evento
 - L'uso della clausola attiva due variabili
 - **NEW** che permette di riferirsi alla tupla appena inserita/aggiornata (in caso di INSERT o UPDATE)
 - **OLD** che permette di riferirsi alla tupla eliminata o prima dell'aggiornamento (DELETE o UPDATE)
- **[WHEN <condizione>]**
 - Specifica la condizione che deve essere controllata dopo che la regola è innescata, MA prima che l'azione è eseguita

ESEMPIO

IMPIEGATO				
Nome	SSN	Stipendio	Num_D	SSN_Super

DIPARTIMENTO			
Nome_D	Num_D	Stip_Totale	SSN_Direttore

- Stip_Totale è un attributo derivato che deve essere aggiornato per mantenere il valore consistente all'interno del dipartimento ← **vincolo**
- Gli eventi che possono causare un cambiamento sono
 1. Inserimento di un nuovo impiegato
 2. Il cambiamento dello stipendio di uno o più impiegati
 3. Il passaggio di un impiegato da un dipartimento ad un altro
 4. L'eliminazione di un impiegato

EVENTO 1

- Bisogna ricalcolare STIP_TOTALE per il dipartimento all'interno del quale è inserito il nuovo impiegato

EVENTO 1

- Bisogna ricalcolare STIP_TOTALE per il dipartimento all'interno del quale è inserito il nuovo impiegato

CREATE TRIGGER StipTotale1

AFTER INSERT ON Impiegato

FOR EACH ROW

WHEN (NEW.Num_D IS NOT NULL)

UPDATE Dipartimento

SET Stip_Totale = Stip_Totale + NEW.Stipendio

WHERE Num_D = NEW.Num_D;

Una INSERT può inserire più di una riga/record

CONDIZIONE

AZIONE: aggiungere lo stip. di ogni nuovo impiegato al totale del dipartimento di appartenenza



NOTA METODOLOGICA: Concettualmente la definizione e implementazione dei vincoli tramite trigger non è di solito particolarmente complicata. È tuttavia importante procedere in modo pedante e accurato in queste due fasi:

1. specifica dei vincoli in fase di analisi dei requisiti e progettazione concettuale/logica;
2. definizione dei trigger (evento-condizione-azione) - questo perché è importante non lasciarsi sfuggire, per esempio, degli eventi che potrebbero violare un vincolo.

Tutto il progetto ovviamente ha come obbiettivo l'implementazione dei casi d'uso. La consistenza dei dati persistenti nella base di dati va vista, in un certo senso, come supporto alla implementazione di detti casi d'uso.

EVENTO 2

- Lo stipendio deve essere ricalcolato nel caso in cui lo stipendio di uno o più impiegati venisse modificato

CREATE TRIGGER StipTotale2

AFTER UPDATE OF Stipendio **ON** Impiegato

FOR EACH ROW

WHEN (**NEW.Num_D IS NOT NULL**)

UPDATE Dipartimento

SET Stip_Totale = Stip_Totale + **NEW.Stipendio - OLD.Stipendio**

WHERE Num_D = **NEW.Num_D**;

← viene aggiornata al totale la diff. tra il nuovo e il vecchio stipendio

EVENTO 3

- Se un impiegato passa da un dipartimento ad un altro, sono due i dipartimenti che devono essere modificati

CREATE TRIGGER StipTotale3

AFTER UPDATE OF Num_D **ON** Impiegato

FOR EACH ROW

BEGIN

UPDATE Dipartimento

SET Stip_Totale = Stip_Totale + **NEW.Stipendio**

WHERE Num_D = **NEW.Num_D**;

UPDATE Dipartimento

SET Stip_Totale = Stip_Totale - **OLD.Stipendio**

WHERE Num_D = **OLD.Num_D**;

END;

} 2 istruzioni SQL UPDATE

EVENTO 4

- Lo stipendio deve essere ricalcolato nel caso in cui un impiegato è licenziato o lascia il dipartimento

CREATE TRIGGER StipTotale4

AFTER DELETE ON Impiegato

FOR EACH ROW

WHEN(**OLD**.Num_D **IS NOT NULL**)

UPDATE Dipartimento

SET Stip_Totale = Stip_Totale – **OLD**.Stipendio

WHERE Num_D = **OLD**.Num_D;

non c'è la tuple NEW

FOR EACH ROW

- È importante notare l'effetto della clausola facoltativa **FOR EACH ROW**
 - La regola viene innescata separatamente per ogni tupla
 - Noto come **trigger a livello di riga**
- Se la clausola venisse tralasciata, il trigger sarebbe noto come **trigger a livello di istruzione** e verrebbe attivato una volta per ogni istruzione
- Esempio: si consideri la seguente operazione di aggiornamento che aumenta del 10% lo stipendio a tutti gli impiegati *del dipartimento n. 5*

UPDATE Impiegato

SET Stipendio = 1.1*Stipendio

WHERE Num_D=5

EVENTO 5

- Vogliamo rilevare tutte le volte che lo stipendio di un impiegato supera lo stipendio del suo supervisore
- Sono molti gli eventi che possono innescare questo evento
 - Inserimento di un nuovo impiegato, modifica di stipendio o modifica di dipartimento per un impiegato
 - Supponiamo che l'azione da eseguire è una procedura SUPER_INFORM che informa il superiore dell'evento

CREATE TRIGGER Super_Inform

BEFORE INSERT OR UPDATE OF Stipendio, SSN_Super **ON** Impiegato

FOR EACH ROW

WHEN(NEW.Stipendio > (SELECT Stipendio FROM Impiegato

WHERE SSN = NEW.SSN_Super))

SUPER_INFORM(NEW.SSN_Super, NEW.SSN)

← viene eseguita una procedura (STORED PROCEDURE)

TORNANDO ALL'ESEMPIO DEL MAGAZZINO

Magazzino (CodArticolo, Descrizione, Quantità)

Ordine (CodOrdine, Data, PIVA, DataInvio)

CompOrdine(CodOrdine*, CodArticolo*, Quantità, Prezzo)

Carrello(CodCarrello, Data, PIVA, Completo)

CompCarrello(CodCarrello*, CodArticolo*, Quantità, Prezzo)

hanno la stessa struttura

- Operazioni che devono essere fatte

Quando il carrello è completo

a) Trasformo il carrello in ordine

1. Creare un nuovo ordine con la struttura del carrello
2. Rimuovere il carrello

b) Aggiornare le scorte nel magazzino

Che operazioni di Manipolazione devo fare?

In che ordine devo fare le operazioni di manipolazione?

TORNANDO ALL'ESEMPIO DEL MAGAZZINO

CREATE TRIGGER creaOrdine

AFTER UPDATE ON CARRELLO OF Completo

WHEN OLD.Completo = 'Incompleto' **AND** NEW.Completo = 'Completo'

FOR EACH ROW

BEGIN

INSERT INTO Ordine (CodO, Data, PIVA)

(SELECT C.CodO, Data, PIVA

FROM Carrello C

WHERE C.CodC = 'NEW.CodC');

Implementazione del vincolo tramite trigger

crea l'ordine

Copia il contenuto del carrello nell'ordine (da CompCarrello a CompOrdine)

INSERT INTO CompOrdine

(SELECT *

FROM CompCarrello C

WHERE C.CodC = NEW.CodC);

DELETE

FROM CompCarrello C

WHERE C.CodC = NEW.CodC;

DELETE

FROM Carrello C

WHERE C.CodC = NEW.CodC;

UPDATE Magazzino M

SET M.Quantità = M.Quantità - (SELECT H.Quantità

FROM CompOrdine H

WHERE H.CodO = 'XYZ' AND H.CodA = M.CodA)

WHERE M.CodA IN (SELECT C.CodA

FROM CompOrdine C

WHERE C.CodO = 'XYZ');

END;

Tutti i prodotti relativi all'ordine in questione

Cancellazioni da Carrello e CompCarrello

INFO SULLA LEZIONE

- Capitolo 8
 - 8.2
- Capitolo 12
 - Fino al 12.2 escluso



In PostgreSQL i trigger devono necessariamente innescare/lanciare una procedura.

```
CREATE OR REPLACE FUNCTION crea_ordine_func()
RETURNS trigger AS $$
BEGIN
  IF OLD.Completo = 'Incompleto' AND NEW.Completo = 'Completo' THEN
    INSERT INTO Ordine (CodO, Data, PIVA)
    SELECT C.CodO, C.Data, C.PIVA
    FROM Carrello C
    WHERE C.CodC = NEW.CodC;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Linguaggio procedurale
di PostgreSQL

PL/pgSQL

```
CREATE TRIGGER creaOrdine
AFTER UPDATE ON Carrello
FOR EACH ROW
EXECUTE FUNCTION crea_ordine_func();
```

Creazione del trigger
che invoca la procedura
qui sopra.

VARIANTE

```
CREATE TRIGGER creaOrdine
AFTER UPDATE ON Carrello
FOR EACH ROW
WHEN (OLD.Completo = 'Incompleto' AND NEW.Completo = 'Completo')
EXECUTE FUNCTION crea_ordine_func();
```

Se utilizziamo questa variante della definizione del trigger,
non è più necessaria la verifica all'interno della procedura
eseguita dalla istruzione IF...THEN...